

Практика 1. Программирование в λ -исчислении

осень 2026

Андрей Стоян, Илья Колегов

Летучка 1

Летучка 1 · 7 минут · Ваше ФИ: _____

1. Запишите с минимумом скобок: $((\lambda x. (\lambda y. (x\ y)))\ a)\ b \equiv$ _____
2. В терме $\lambda_1 x. (\lambda_2 x. \lambda_3 y. x)\ (x\ y)$ для каждого вхождения переменной слева направо напишите номер её связывателя или «св», если она свободна: x ____ x ____ y ____
3. Поставьте $=_\alpha$ или $\neq_\alpha$: $\lambda x\ y. x\ y$ ____ $\lambda a\ b. a\ b$ $\lambda x. y\ x$ ____ $\lambda z. y\ z$ $\lambda x. x\ y$ ____ $\lambda y. y\ x$
4. Сделайте один β -шаг: $(\lambda x. x\ x)\ (\lambda y. y) \rightsquigarrow$ _____

Метаинформация о курсе

Зачем этот курс

- Мы возьмём простейший язык — λ -исчисление — и построим программирование заново
- $\Rightarrow$ Вы научитесь описывать намерение так, чтобы его проверяла машина
- $\Rightarrow$ Научитесь смотреть на каждый новый язык как на набор знакомых концепций
- $\Rightarrow$ И перестанете считать языковые инструменты магией — потому что соберёте их сами
- **NB** Сегодня за час мы соберём из одних только функций: пары, логические значения и числа

Как устроена неделя

- **Δ Лекция** — новая тема
- **Δ Практика**
 - пишем летучку, она отрывается и сдаётся
 - мы разговариваем и решаем задачи, как и на зачётных сессиях
 - *присутствуйте*, используйте это время максимально, не откладывая на потом
 - ноутбуками не пользуемся
 - слайды и материалы будут — имеет смысл конспектировать только свои мысли
- **Δ Домашка**
 - выдаётся каждую неделю на неделю
 - базовое закрепление и подготовка к зачётным сессиям
 - рассчитана на решения “вручную”
 - если вы всё поняли, решается очень быстро, в действительности медленнее — это нормально
- **Δ Ревью**
 - после дедлайна бот рассылает решение случайно между одноклассниками
 - помощь другому — хороший способ научиться самому

Зачёт по практике

- **Летучки** - нужно сдать 12 из 14, правильность не учитывается
- **Домашки** - нужно сдать 12 из 14 на минимальный порог
- **Очные сессии**
 - нужно сдать 3
 - 20 минут решаем задачку маркером на доске и обсуждаем
- Старайтесь не терять темп, но будет достаточно возможностей дозаккрыть долги
- Зачёт по практике необходим, чтобы получить допуск на экзамен
- Оценка за курс формируется на экзамене

Обозначения на слайдах

- Δ определение
- e.g. пример
- $\Rightarrow$ следствие, вывод
-  — задание
-  как это выглядит на практике
- *i* для общего кругозора
- **NB** обратите внимание
- $\lessgtr$ — разбор

λ -исчисление

Что значит «задать язык»

- **Δ Синтаксис** — как записывать программы на этом языке
- **Δ Семантика** — как исполнять программы на этом языке

Синтаксис

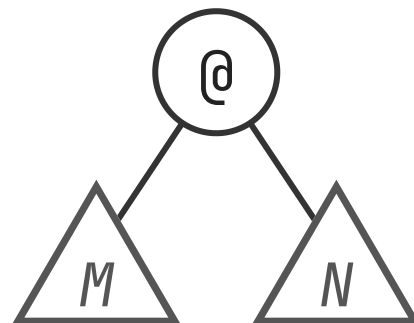
Формальный синтаксис термов

$$\Lambda ::= V \mid (\Lambda \ \Lambda) \mid (\lambda V. \Lambda)$$

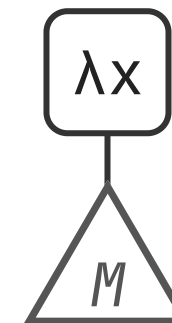
Переменная x



Аппликация $(M \ N)$



Абстракция $(\lambda x. M)$



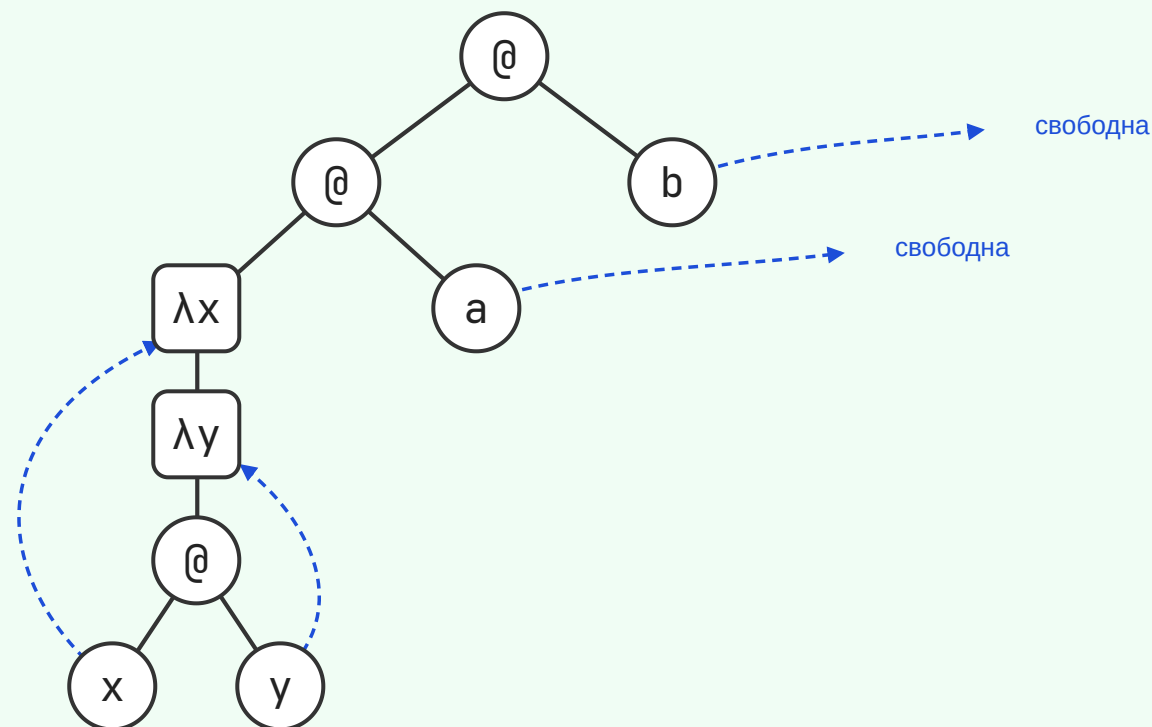
- Определение индуктивное, поэтому каждый терм — дерево
- Скобки нужны только строковой записи, в дереве их нет

Дерево абстрактного синтаксиса (AST)

✎ Терм записан со всеми скобками: $((\lambda x. (\lambda y. (x\ y)))\ a)\ b$. Нарисуйте его дерево и запишите терм с минимумом скобок. Отметьте на дереве, какая λ связывает каждую переменную. Есть ли здесь свободные переменные?

⚡ Решение.

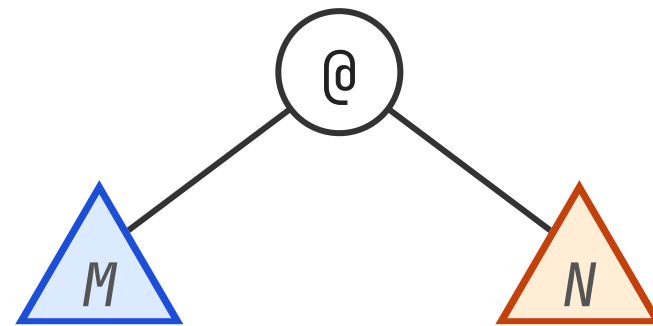
Минимальная запись $(\lambda x\ y. x\ y)\ a\ b$; x связана внешней λ , y — внутренней, а a и b свободны.



Строковая запись и дерево

Каждая пара скобок в записи — один узел дерева. Внешние скобки — аппликация: слева функция, справа аргумент.

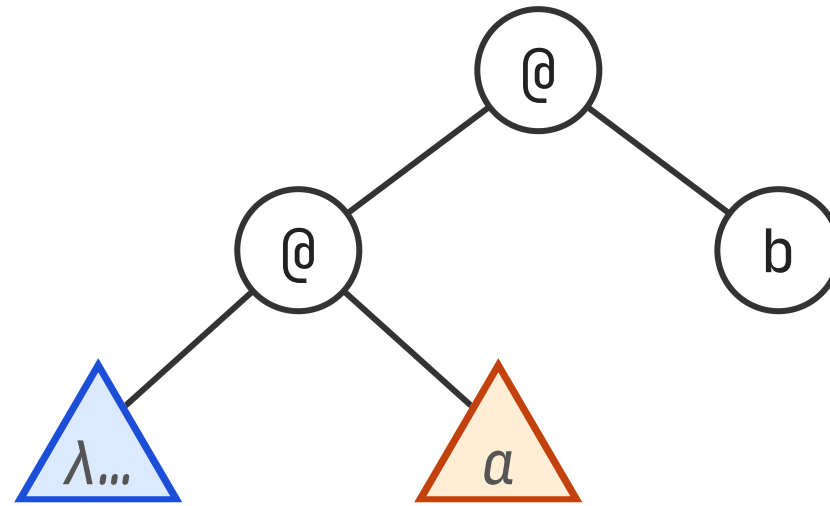
$((\lambda x. (\lambda y. (x y))) a) b$



Строковая запись и дерево

Снова аппликация: функция $(\lambda x. (\lambda y. (x y)))$ и её аргумент a .

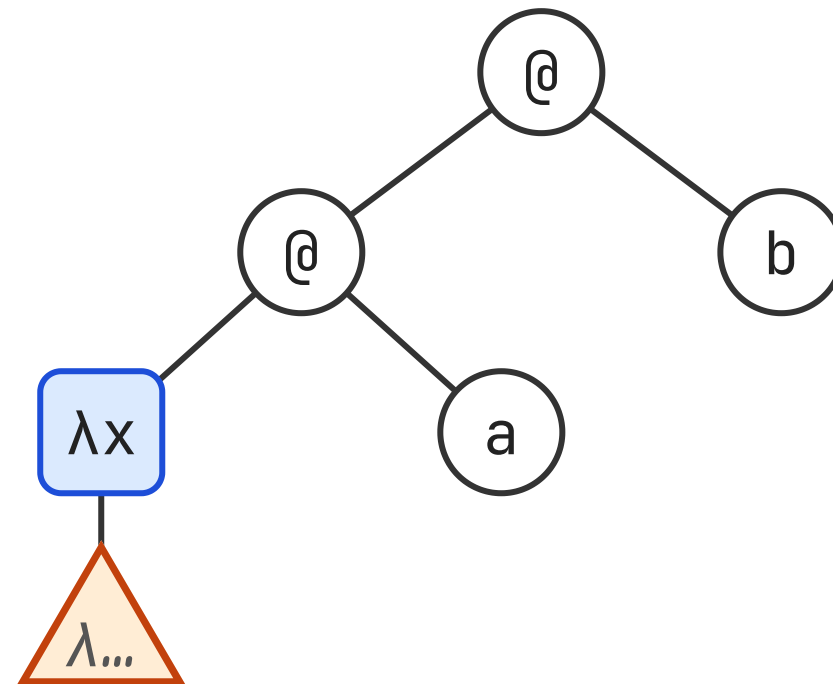
$((\lambda x. (\lambda y. (x y))) a) b$



Строковая запись и дерево

Скобки вокруг λx — узел абстракции. У него один ребёнок: тело $(\lambda y. (x y))$.

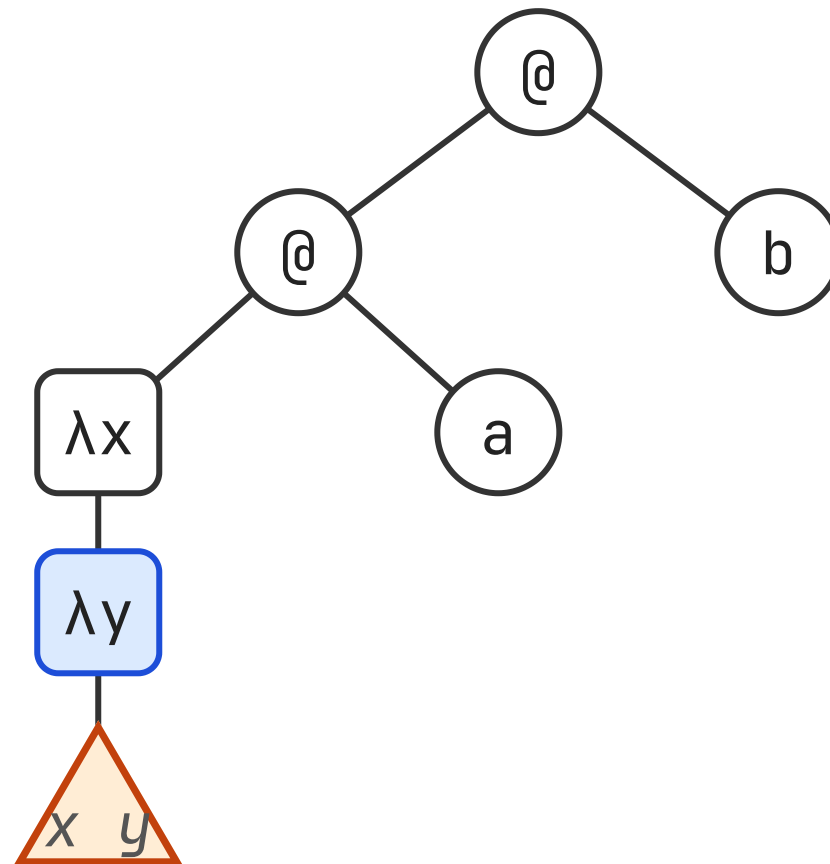
$((\lambda x. (\lambda y. (x y))) a) b$



Строковая запись и дерево

Ещё одна абстракция: λy и её тело $(x\ y)$.

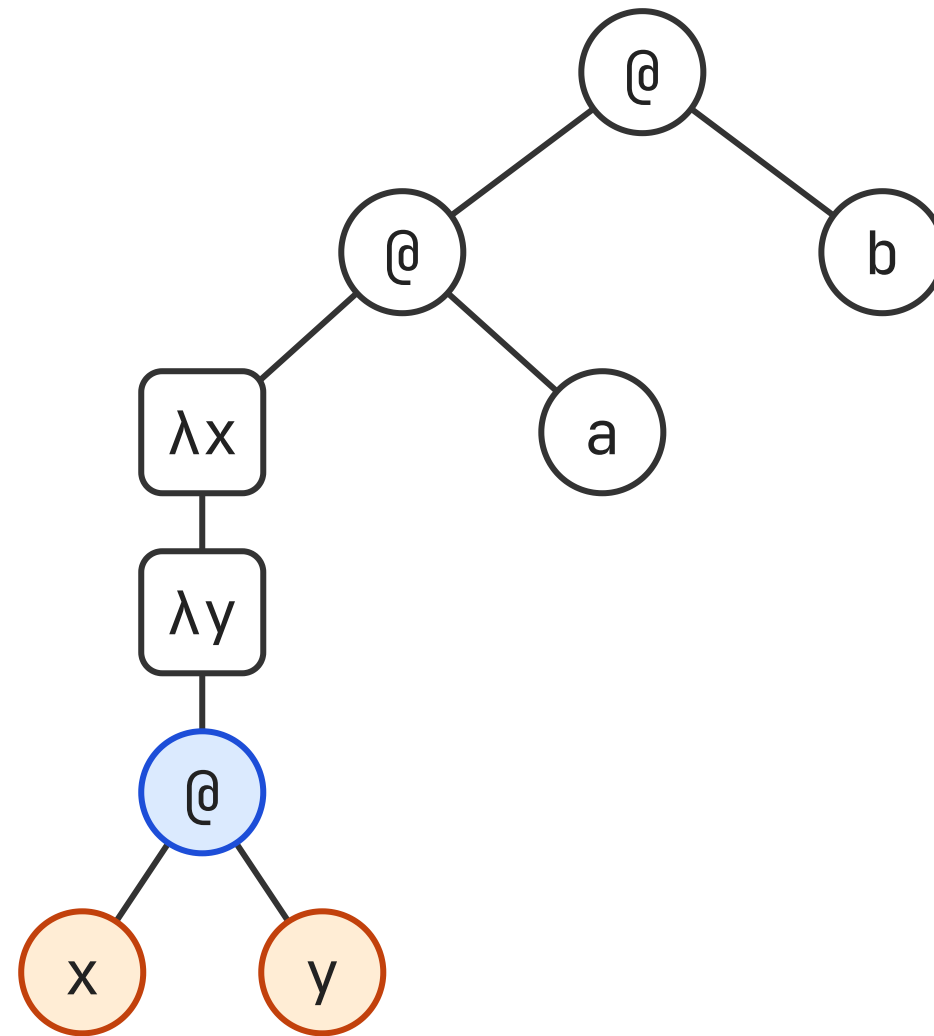
$((((\lambda x. (\lambda y. (x\ y)))\ a)\ b)$



Строковая запись и дерево

Последняя пара скобок — аппликация переменной x к переменной y . Дерево готово.

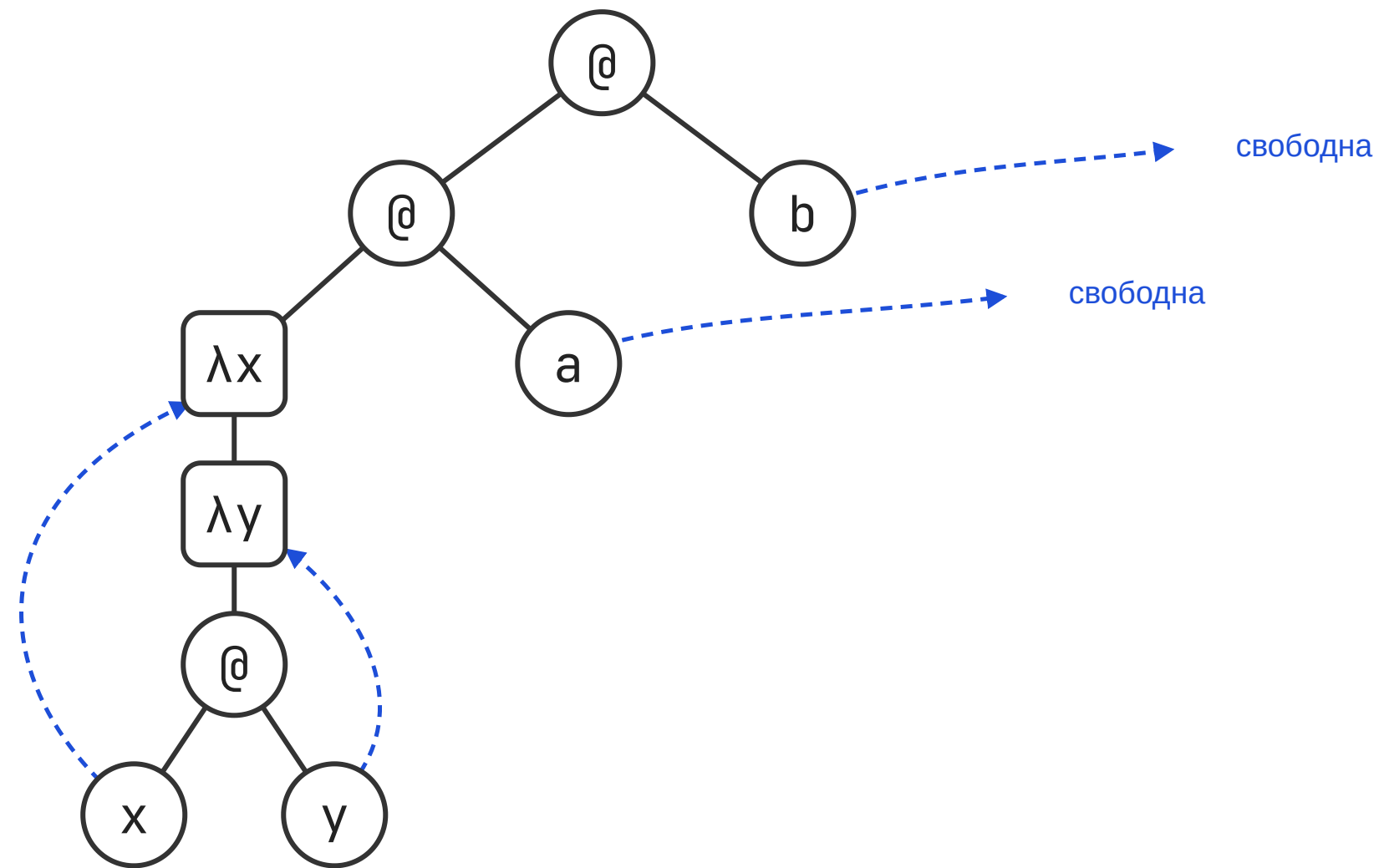
$(((\lambda x. (\lambda y. (x\ y)))\ a)\ b)$



Строковая запись и дерево

- Можем убрать скобки, постановка которых не влияет на форму дерева
- Стрелки показывают, какая λ связывает каждую переменную; у a и b связывателя в терме нет

$(\lambda x y. x y) a b$

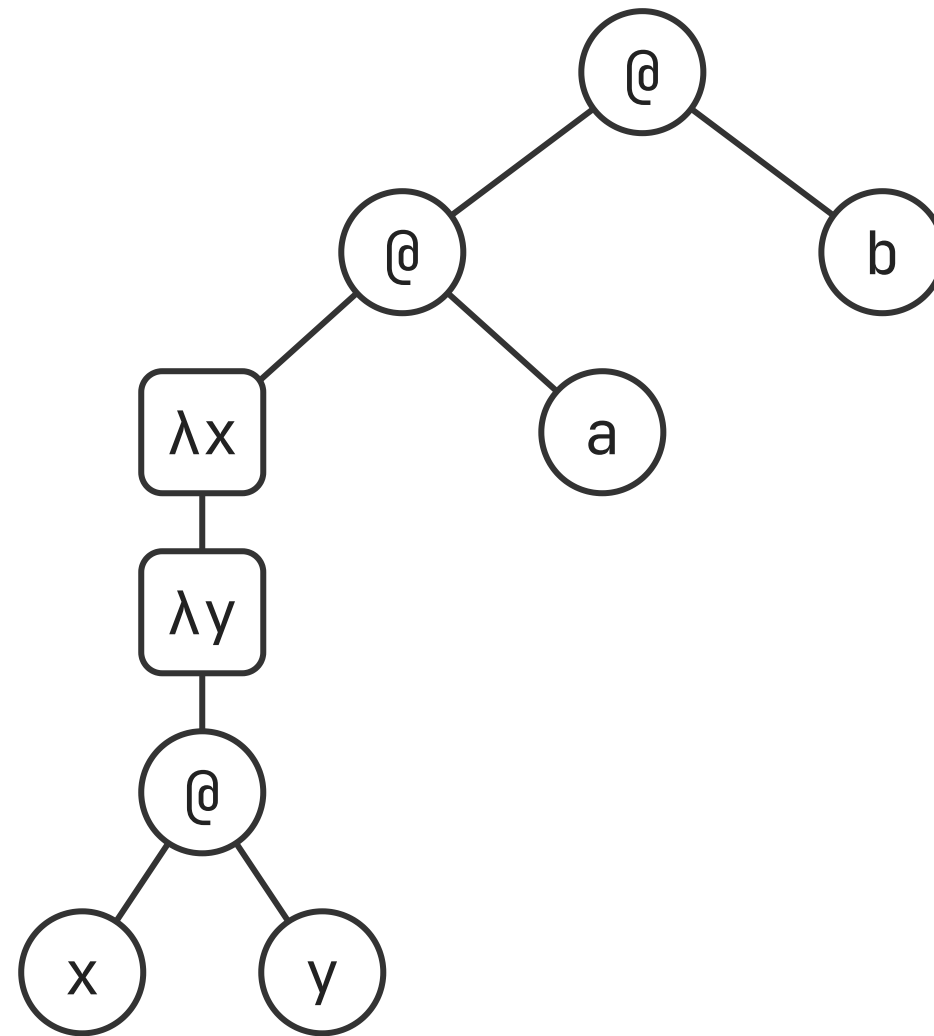


Семантика

Шаг β -редукции

✎ **Устно:** Программа — это терм. Где в нём «вызов функции», который можно выполнить?

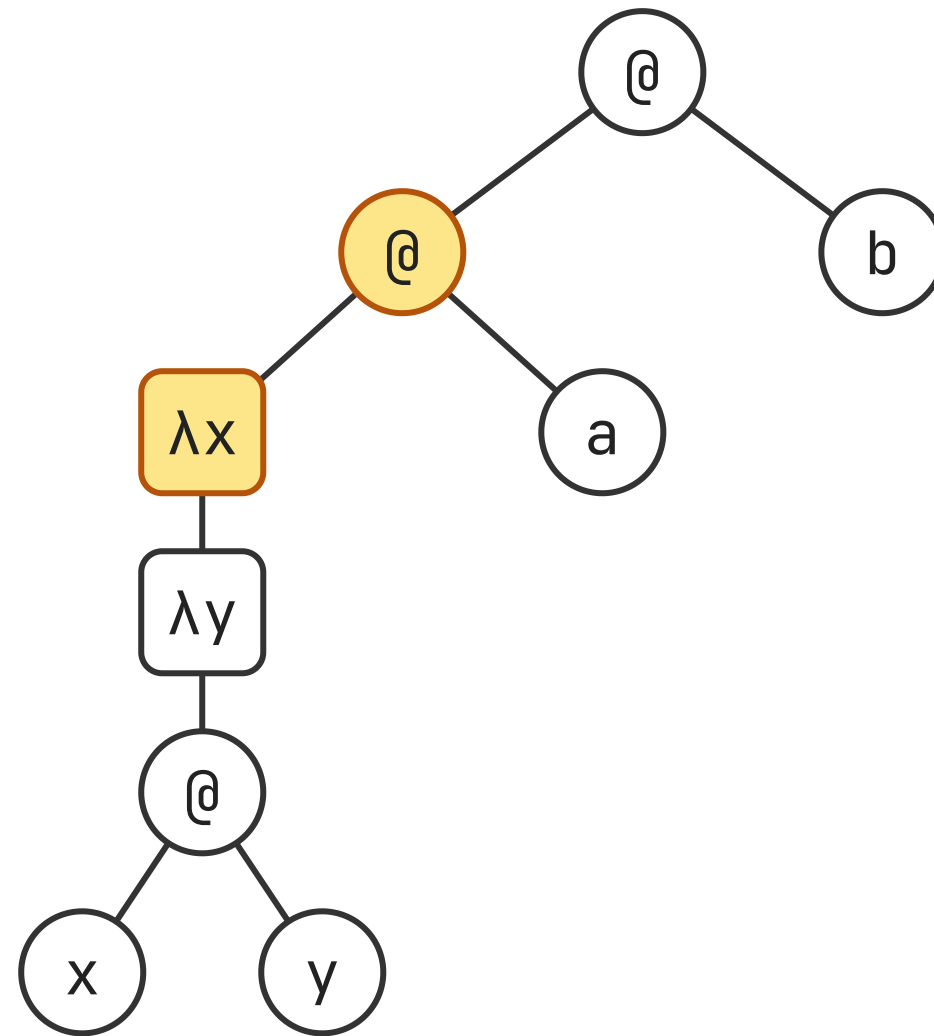
$(\lambda x y. x y) a b$



Шаг β -редукции

- Δ **Редекс** — аппликация, слева от которой стоит абстракция. На дереве это узел @, у которого левый ребёнок — λ

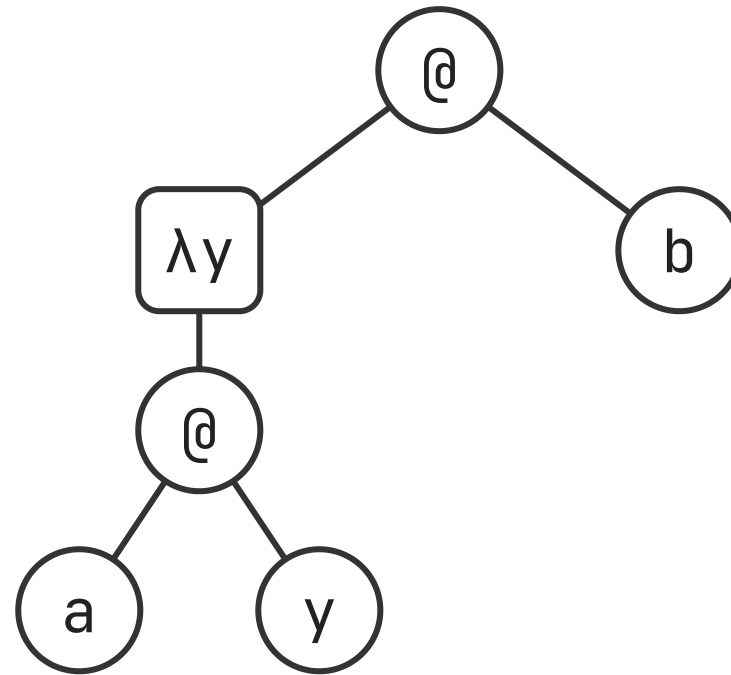
$(\lambda x \ y. \ x \ y) \ a \ b$



Шаг β -редукции

Аргумент a занял место x , узлы $@$ и λx исчезли.

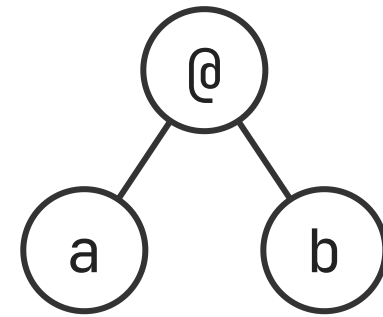
$(\lambda y. a y) b$



Шаг β -редукции

Второй шаг — так же. Редексов не осталось, это нормальная форма.

a b



Запишем ту же редукцию строками

 **Устно:** Запишите редукцию $(\lambda x y. x y) a b$ цепочкой преобразований.

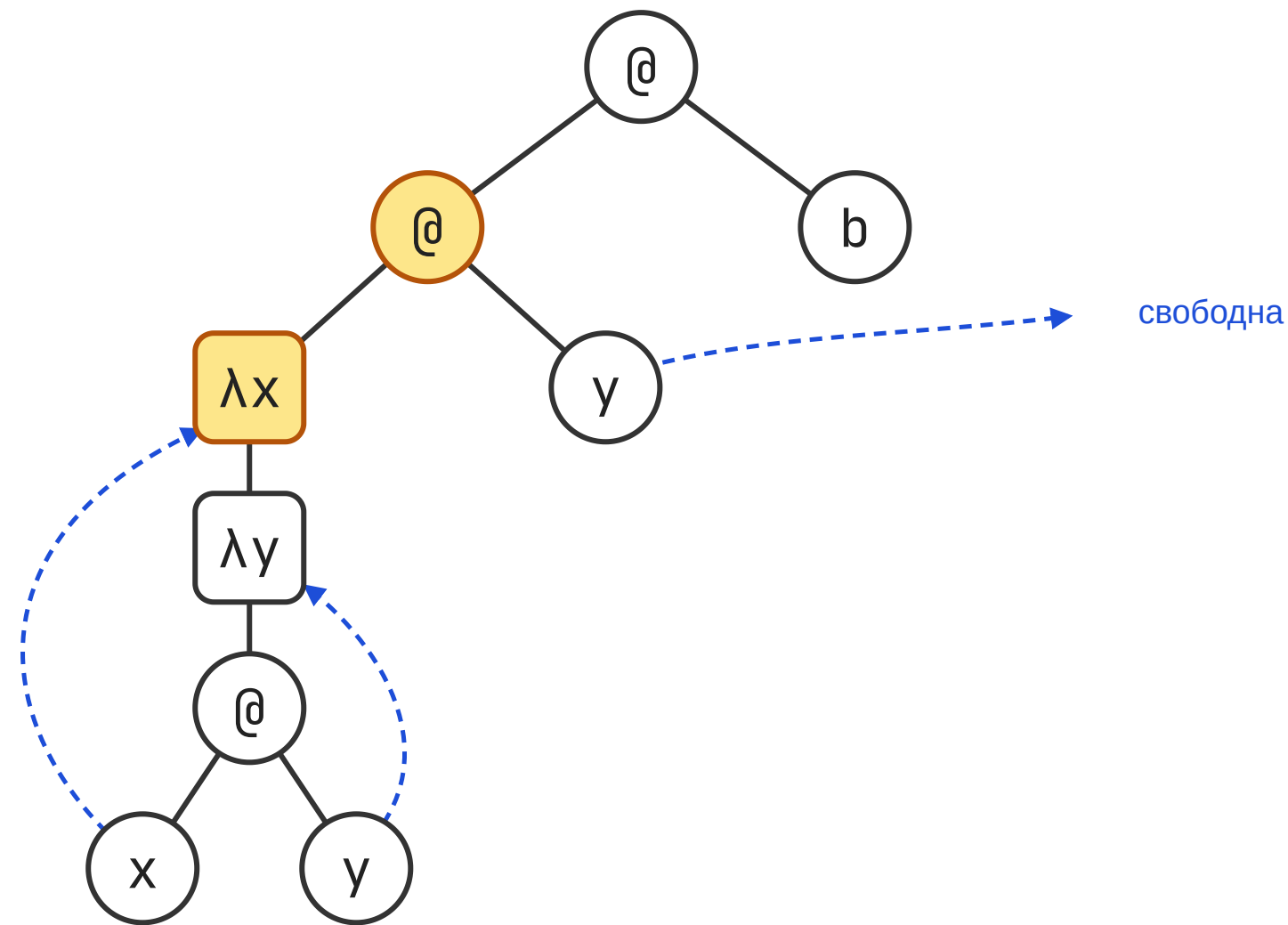
 **Решение.**

1. $(\lambda x y. x y) a b$
2. $\rightsquigarrow_{\beta} (\lambda y. a y) b$
3. $\rightsquigarrow_{\beta} a b$

Случайный захват

✎ **Устно:** Тот же терм, только аргумент теперь y . Сделайте β -шаг в уме — что получилось?

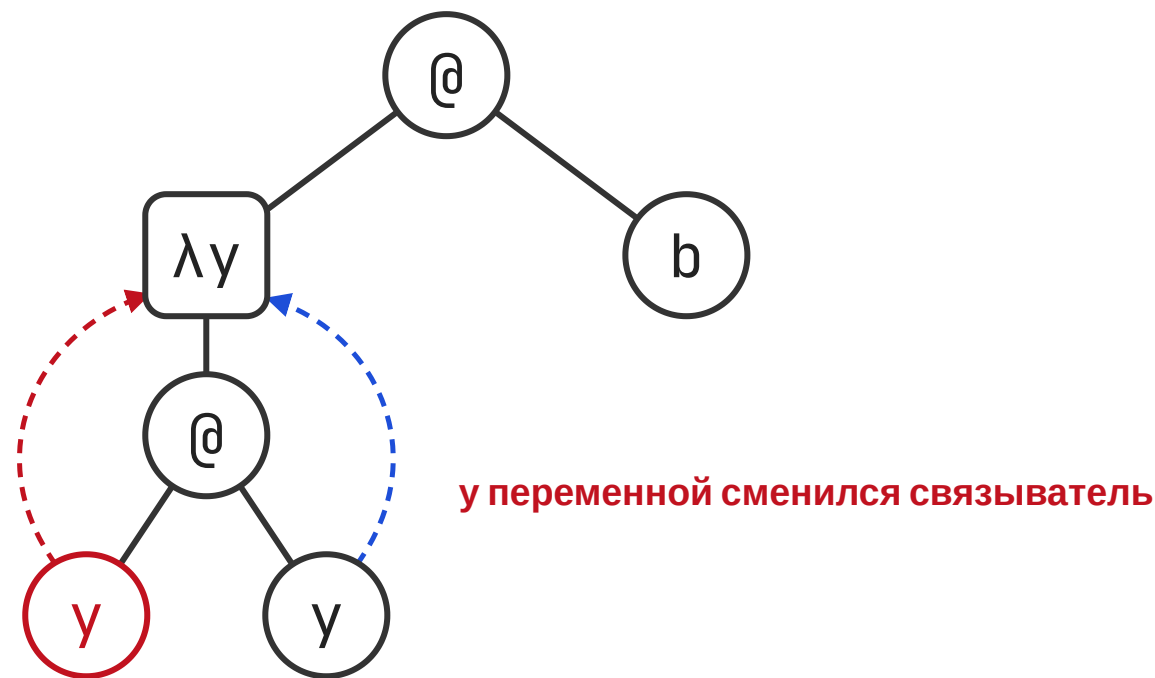
$(\lambda x y. x y) y b$



Случайный захват

Пересадили «в лоб» — получили $(\lambda y. y y) b$. Но стрелка от нового y перекинулась на λy : переменную захватили.

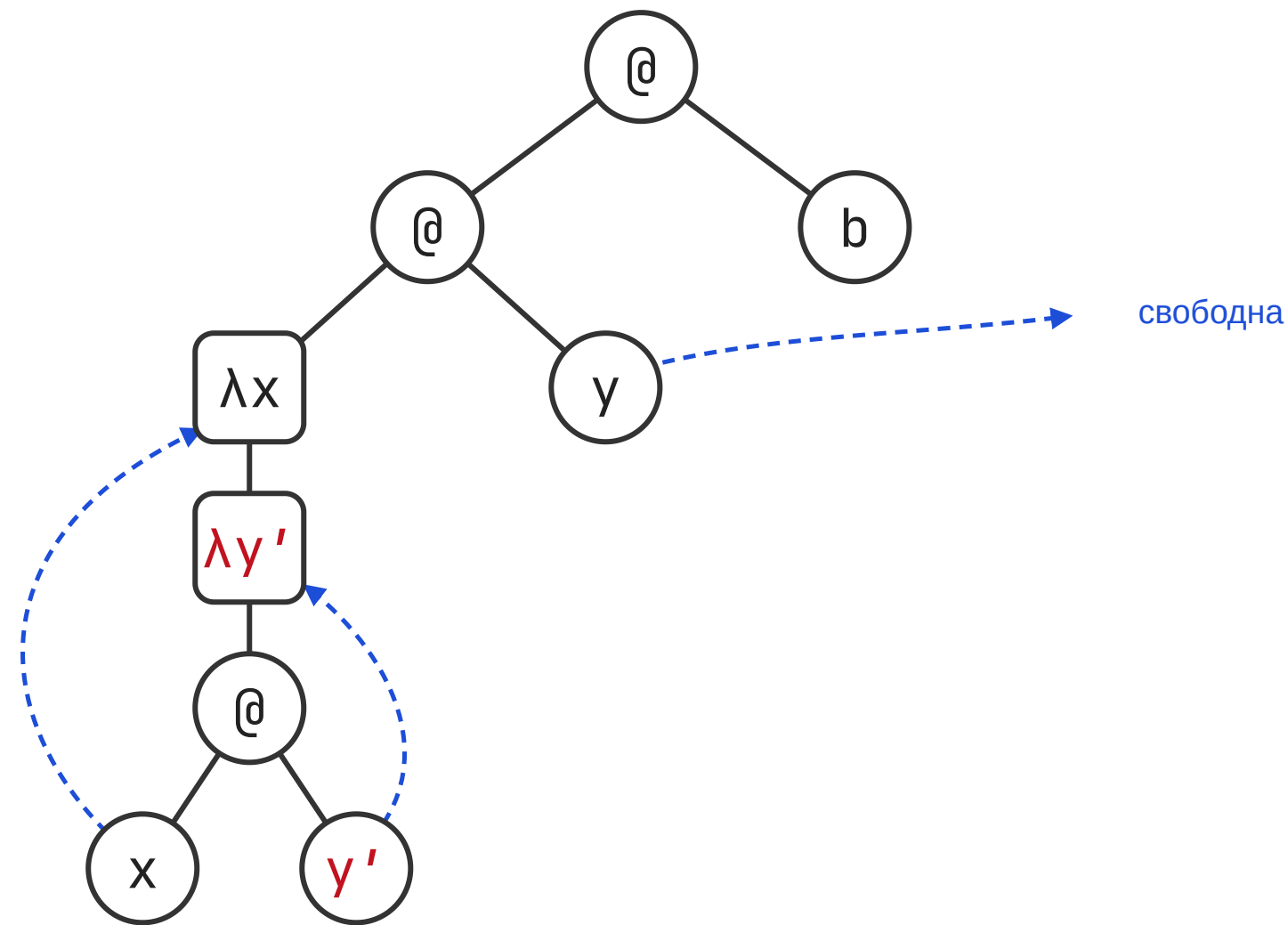
$(\lambda y. y y) b$



Случайный захват

Откатываемся. Сначала переименуем связанную переменную — терм от этого не меняется, он α -эквивалентен.

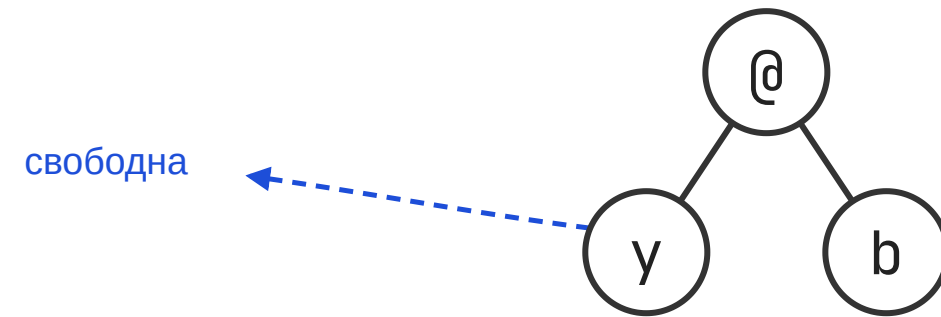
$(\lambda x y'. x y') y b$



Случайный захват

Теперь оба шага проходят спокойно, и свободная y остаётся свободной.

y b



Подстановка

$$[x \mapsto N] x \equiv N$$

$$[x \mapsto N] y \equiv y$$

$$[x \mapsto N] (P Q) \equiv ([x \mapsto N] P) ([x \mapsto N] Q)$$

$$[x \mapsto N] \lambda x. P \equiv \lambda x. P$$

$$[x \mapsto N] \lambda y. P \equiv \lambda y. [x \mapsto N] P \quad y \notin FV(N)$$

$$[x \mapsto N] \lambda y. P \equiv \lambda y'. [x \mapsto N] ([y \mapsto y'] P) \quad y \in FV(N)$$

 Выполните подстановки:

$$(a) [y \mapsto x] x (\lambda y. y z) \quad (б) [z \mapsto x] x (\lambda y. y z) \quad (в) [z \mapsto x y] x (\lambda y. y z)$$

 **Решение.**

$$(a) x (\lambda y. y z) \text{ — под } \lambda y \text{ менять нечего} \quad (б) x (\lambda y. y x) \text{ — проход под связыватель} \quad (в) \\ x (\lambda y'. y' (x y)) \text{ — правило 6}$$

Когда два терма — одно и то же

✎ Эквивалентны ли термы, и если да, то в каком смысле — синтаксически, α , β или η ?

(а) x и x (б) x и y (в) $\lambda x. x$ и $\lambda y. y$ (г) $\lambda x. x$ и $(\lambda y. y) (\lambda x. x)$ (д) $\lambda x y. x y$ и $\lambda x. x$

✚ Решение.

(а) это один и тот же терм, равенство синтаксическое (б) не эквивалентны (в) α : отличаются только именем связанной переменной (г) β : правый терм редуцируется в левый (д) η : $\lambda y. x y$ и x ведут себя одинаково на любом аргументе

Кодирование данных

Пары

Записи, кортежи, пары

Одна сущность часто описывается несколькими свойствами: котик — это имя и мурлычность.

```
class Cat:
    def __init__(self, name, murlycity):
        self.name = name
        self.murlycity = murlycity
```

Чего мы хотим от такой структуры данных:

- **Δ Конструктор** создаёт новое значение: `barsik = Cat("barsik", 4)`
- **Δ Элиминаторы** достают информацию из значения: `barsik.murlycity`
- **NB** Мутаторы (`barsik.murlycity += 3`) не используем: вместо изменения — новое значение `Cat(barsik.name, barsik.murlycity + 3)`

Спецификация пары

API пары — терм-конструктор **pair**, элиминаторы **fst** и **snd**

Законы пар

1. $\text{fst } (\text{pair } N \ M) =_{\beta} N$
2. $\text{snd } (\text{pair } N \ M) =_{\beta} M$

✎ **Устно:** Запишите **swap**, меняющий компоненты пары местами, через **pair**, **fst** и **snd**. Проверьте по законам: $\text{fst } (\text{swap } (\text{pair } N \ M)) =_{\beta} M$.

⚡ **Решение.**

$$\text{swap} := \lambda p. \text{pair } (\text{snd } p) (\text{fst } p)$$
$$\text{fst } (\text{swap } (\text{pair } N \ M)) =_{\beta} \text{fst } (\text{pair } (\text{snd } (\text{pair } N \ M)) \ \dots) =_{\beta} \text{snd } (\text{pair } N \ M) =_{\beta} M$$

Пары в λ -исчислении

- В λ -исчислении нет ничего, кроме функций. Где хранить два значения?

Конструктор принимает два значения и функцию, которой отдаёт оба:

- **pair** $:= \lambda x y. \lambda f. f\ x\ y$ — любой терм вида $\lambda f. f\ N\ M$ можно считать парой

Элиминаторы:

- **fst** $:= \lambda p. p\ \mathbf{K}$
- **snd** $:= \lambda p. p\ \mathbf{K}_*$

Проверка законов пар

pair $:= \lambda x y. \lambda f. f x y$ **fst** $:= \lambda p. p \mathbf{K}$ **snd** $:= \lambda p. p \mathbf{K}_*$

✎ Проверьте, что второй закон пар выполняется.

✂ *Решение.*

$$\begin{aligned} & \mathbf{snd} (\mathbf{pair} N M) \\ \rightsquigarrow_{\mathbf{pair}} & \mathbf{snd} ((\lambda x y. \lambda f. f x y) N M) \\ \rightsquigarrow_{\beta} & \mathbf{snd} (\lambda f. f N M) \\ \rightsquigarrow_{\mathbf{snd}} & (\lambda p. p \mathbf{K}_*) (\lambda f. f N M) \\ \rightsquigarrow_{\beta} & (\lambda f. f N M) \mathbf{K}_* \\ \rightsquigarrow_{\beta} & \mathbf{K}_* N M \rightarrow M \end{aligned}$$

Итого $\mathbf{snd} (\mathbf{pair} N M) \rightarrow M$ — дальше пользуемся этим как правилом «большого шага».

Логические величины

Спецификация логических величин

API логических величин — конструкторы-константы **true** и **false**, элиминатор **if**

 **Устно:** Сформулируйте законы логических величин по аналогии с законами пар.

 **Решение.**

1. **if true** $T \ E =_{\beta} T$

2. **if false** $T \ E =_{\beta} E$

Логические величины в λ -исчислении

Конструкторы принимают две ветки и выбирают из них свою:

- **true** $:= \lambda t e. t$ — это **K**
- **false** $:= \lambda t e. e$ — это **K_{*}**

Элиминатор — выбор уже сделан значением, осталось применить:

- **if** $:= \lambda b t e. b t e$, или просто **if** $:= \mathbf{I}$

Операции над логическими величинами: отрицание

✎ Закодируйте **not** через **if** по таблице истинности:

b	$\neg b$
true	false
false	true

⚡ *Решение.*

not := $\lambda b. \text{if } b \text{ false true}$

✎ **Устно:** А **and**? Подсказка: если a истинно, ответ зависит только от b .

⚡ *Решение.*

and := $\lambda a b. \text{if } a \text{ b false}$

Натуральные числа

Натуральные числа $\mathbb{N} := \{0, 1, 2, 3, \dots\}$

API натуральных чисел

- Первая аксиома Пеано: ноль является натуральным числом (**0**)
- Вторая аксиома Пеано: число, следующее за натуральным, тоже натуральное (**suc n**)
- Элиминатор: воспользоваться натуральным числом — что-то сделать n раз, **natElim**

Законы для натуральных чисел

1. **natElim 0 F $Z =_{\beta} Z$**
2. **natElim (suc N) F $Z =_{\beta} F$ (natElim N F Z)**

Законы natElim в действии

Число **3** — это `suc (suc (suc 0))`. Посчитаем `natElim 3 F Z` одними законами, не зная, как числа устроены.

natElim 3 F Z

ЦЕПОЧКА
natElim 3 F Z

Законы natElim в действии

3 — это **suc 2**: второй закон снимает один **suc** и применяет F один раз.

F (natElim 2 F Z)

ЦЕПОЧКА

natElim 3 F Z

$\rightsquigarrow_3$ **natElim (suc 2) F Z**

$=_\beta$ F (**natElim 2 F Z**) — закон 2

Законы natElim в действии

Ещё раз: **2** — это **suc 1**.

$F (F (\text{natElim } 1 \ F \ Z))$

ЦЕПОЧКА

$\text{natElim } 3 \ F \ Z$

$\rightsquigarrow_3 \text{natElim } (\text{suc } 2) \ F \ Z$

$=_\beta F (\text{natElim } 2 \ F \ Z) — \text{закон } 2$

$\rightsquigarrow_2 F (\text{natElim } (\text{suc } 1) \ F \ Z)$

$=_\beta F (F (\text{natElim } 1 \ F \ Z)) — \text{закон } 2$

Законы natElim в действии

И ещё: **1** — это `suc 0`.

$F (F (F (\text{natElim } 0 \ F \ Z)))$

ЦЕПОЧКА

natElim 3 F Z

$\rightsquigarrow_3 \text{natElim (suc 2) } F \ Z$

$=_\beta F (\text{natElim 2 } F \ Z) - \text{закон 2}$

$\rightsquigarrow_2 F (\text{natElim (suc 1) } F \ Z)$

$=_\beta F (F (\text{natElim 1 } F \ Z)) - \text{закон 2}$

$\rightsquigarrow_1 F (F (\text{natElim (suc 0) } F \ Z))$

$=_\beta F (F (F (\text{natElim 0 } F \ Z))) - \text{закон 2}$

Законы natElim в действии

Дошли до **0** — первый закон возвращает Z . F применили столько раз, сколько **suc** в числе.

$F (F (F Z))$

ЦЕПОЧКА

$\text{natElim } 3 \ F \ Z$

$\rightsquigarrow_3 \text{ natElim } (\text{suc } 2) \ F \ Z$

$=_\beta \ F (\text{natElim } 2 \ F \ Z) - \text{закон } 2$

$\rightsquigarrow_2 \ F (\text{natElim } (\text{suc } 1) \ F \ Z)$

$=_\beta \ F (F (\text{natElim } 1 \ F \ Z)) - \text{закон } 2$

$\rightsquigarrow_1 \ F (F (\text{natElim } (\text{suc } 0) \ F \ Z))$

$=_\beta \ F (F (F (\text{natElim } 0 \ F \ Z))) - \text{закон } 2$

$=_\beta \ F (F (F Z)) - \text{закон } 1$

Операции над числами Чёрча: чётность

✎ **Факультативно:** Закодируйте **isEven** одними законами. Рецепт:

1. базовые случаи — **isEven 0** $=_{\beta}$ **true**, **isEven 1** $=_{\beta}$ **false**, **isEven 2** $=_{\beta}$ **true**, ...
2. как воспользоваться числом — **natElim 3 F Z** $\rightarrow F (F (F Z))$
3. подобрать F и Z

⚡ **Решение.**

Состояние — пара «ответ сейчас, ответ на следующем шаге»: **start** := **pair true false**, шаг — **swap** из разбора пар, **isEven** := $\lambda n. \text{fst } (\text{natElim } n \text{ swap start})$

⚡ **Решение.**

Состояние — одно логическое значение: **isEven** := $\lambda n. \text{natElim } n \text{ not true}$

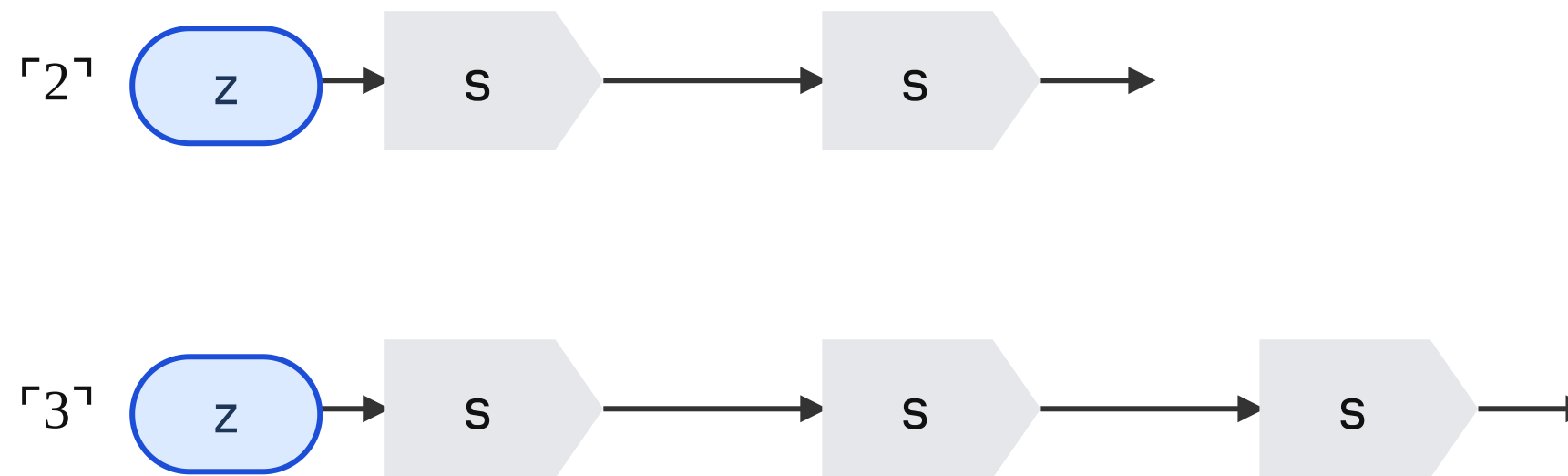
Числа Чёрча

Число знает, сколько раз применить шаг s к начальному значению z :

$$\mathbf{0} := \lambda s z. z \quad \mathbf{1} := \lambda s z. s z \quad \mathbf{2} := \lambda s z. s (s z) \quad \dots$$

Элиминатор — снова применить значение к аргументам: **natElim** $:= \lambda n s z. n s z$, то есть **I**.

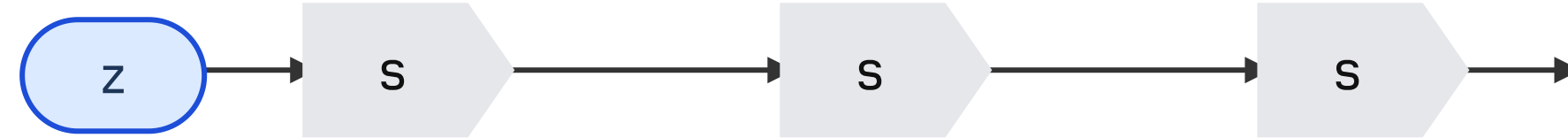
Числа по Пеано — те же самые: **suc** (**suc** (**suc** **0**)) $\rightarrow \lambda s z. s (s (s z))$, и законы выполняются.



Как работают числа Чёрча

Число — конвейер; **natElim** ставит в него трансформации и вход и запускает.

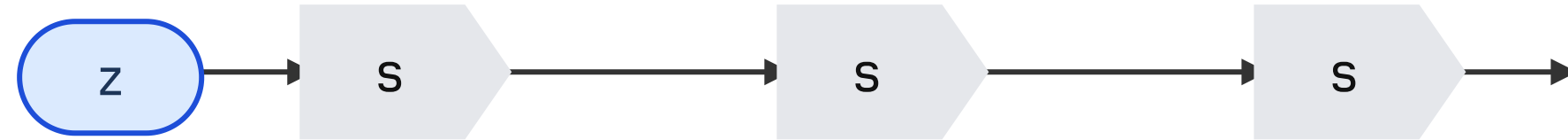
3 := $\lambda s z. s (s (s z))$



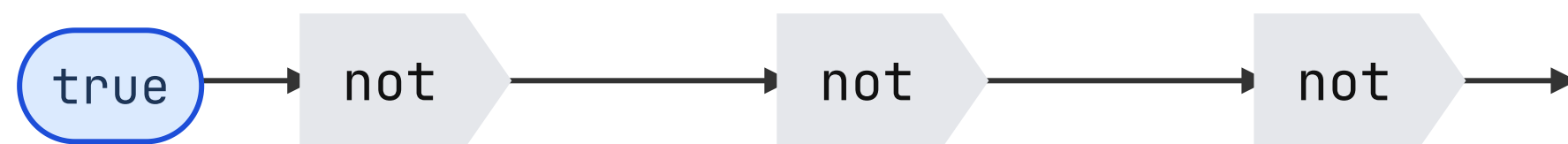
Как работают числа Чёрча

Число — конвейер; **natElim** ставит в него трансформации и вход и запускает.

3 := $\lambda s z. s (s (s z))$



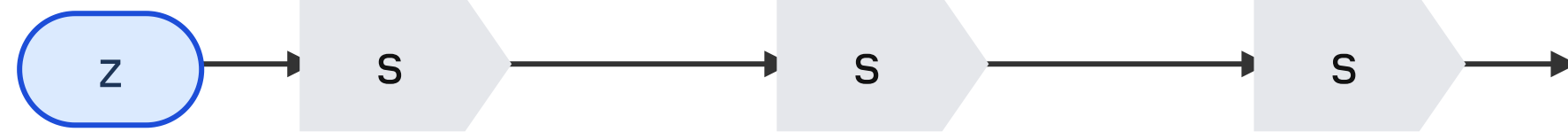
natElim 3 not true $\Rightarrow$ **3 not true**



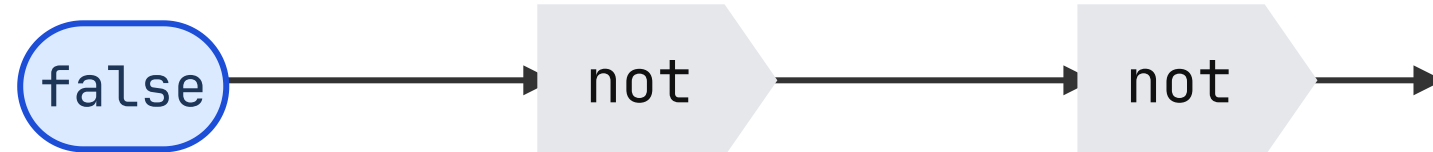
Как работают числа Чёрча

Число — конвейер; **natElim** ставит в него трансформации и вход и запускает.

3 := $\lambda s z. s (s (s z))$



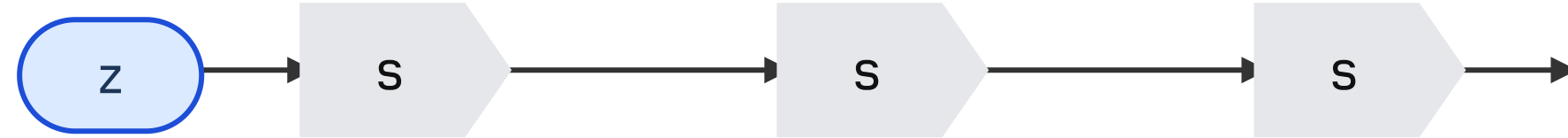
natElim 3 not true $\Rightarrow$ **3 not true** $\Rightarrow$ **not (not false)**



Как работают числа Чёрча

Число — конвейер; **natElim** ставит в него трансформации и вход и запускает.

3 := $\lambda s z. s (s (s z))$



natElim 3 not true $\Rightarrow$ **3 not true** $\Rightarrow$ **not (not false)** $\Rightarrow$ **not true**



Как работают числа Чёрча

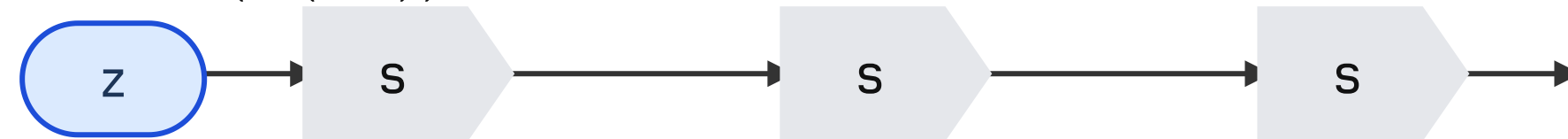
Число — конвейер; **natElim** ставит в него трансформации и вход и запускает.

✎ **Устно:** Трансформаций не осталось: **natElim 3 not true** $\rightarrow$ **false**. Какая семантика у терма **natElim n not true**?

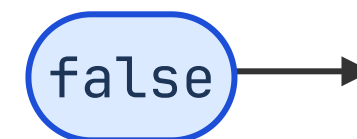
⚡ **Решение.**

Чётность: это **isEven** с прошлого задания, $\lambda n. \text{natElim } n \text{ not true}$

3 := $\lambda s z. s (s (s z))$



natElim 3 not true $\rightarrow$ **3 not true** $\rightarrow$ **not (not false)** $\rightarrow$ **not true** $\rightarrow$ **false**



Следующее число

✎ Запишите **suc**: результат должен применять s на один раз больше, чем n . Подсказка: воспользуйтесь числом n через **natElim**.



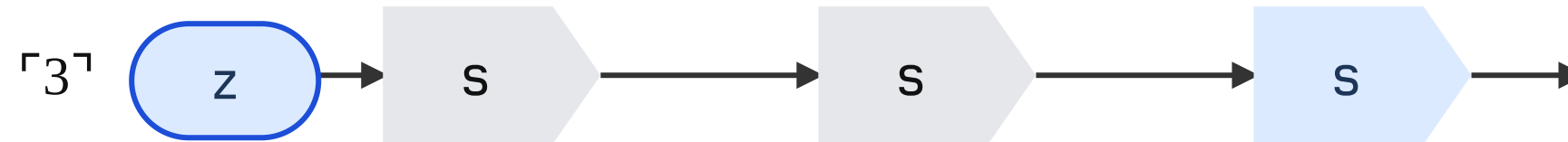
Следующее число

✎ Запишите **suc**: результат должен применять s на один раз больше, чем n . Подсказка: воспользуйтесь числом n через **natElim**.

⚡ **Решение.**

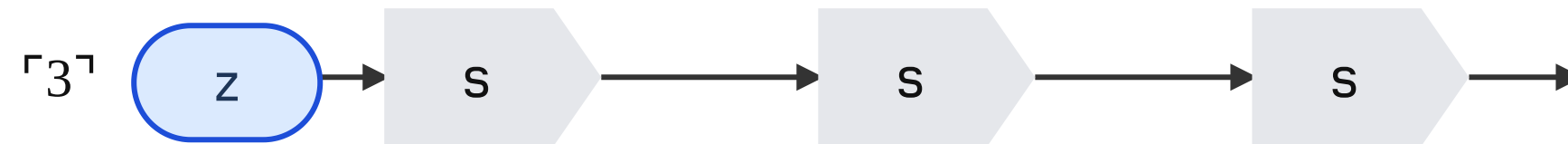
Новое s — снаружи: **suc** $:= \lambda n. \lambda s z. s \text{ (natElim } n s z)$.

Можно и изнутри: $\lambda n. \lambda s z. \text{natElim } n s (s z)$ — цепочка та же.



Операции над числами Чёрча: сложение

✎ Закодируйте сложение.



Операции над числами Чёрча: сложение

✎ Закодируйте сложение.

⚡ *Решение.*

plus := $\lambda n m. \lambda s z. \mathbf{natElim} \ n \ s \ (\mathbf{natElim} \ m \ s \ z)$.

Или n раз увеличить m на единицу: **plus** := $\lambda n m. \mathbf{natElim} \ n \ \mathbf{suc} \ m$



⌈3⌉, потом ⌈2⌉ = ⌈5⌉

Что вы теперь умеете

с лекции	I	K	K _*	S	B
пары	pair	fst	snd		
логика	true	false	if	not	
числа	0	suc	plus		

- Мы задали простейший язык программирования — описали синтаксис и семантику λ -исчисления
 - программы (термы) — деревья правильного вида
 - вычисление — переписывание деревьев
- Мы спроектировали первые структуры данных в λ -исчислении
 - начинали с API — набор конструкторов и элиминаторов
 - записывали законы, определяющие поведение конструкторов и элиминаторов
 - конструировали термы, удовлетворяющие законам

 **Устно:** Закодируйте аналогичным образом структуры данных в вашем любимом языке программирования.

Материалы

Каррирование

Эквивалентен ли код?

$\mathbf{K}_* := \lambda x y. y$

```
def k_star(x, y):  
    return y
```

Нет. $\mathbf{K}_* q \rightsquigarrow \lambda y. y$ — получилась функция, а $\mathbf{k_star}(q)$ — ошибка «missing 1 required positional argument». Правильное прочтение: $\lambda x y. y \equiv \lambda x. \lambda y. y$, то есть функция, которая игнорирует аргумент и возвращает другую функцию:

```
def k_star(x):  
    def helper(y):  
        return y  
    return helper
```

- **Δ Каррирование** превращает функцию многих аргументов в набор вложенных функций, каждая из которых берёт часть аргументов. В ООП методы каррированы: часть «аргументов» приходит через конструктор

Замыкания и функции высших порядков

Закодируем $\mathbf{K} := \lambda x. \lambda y. x$:

```
def k(x):  
    def helper(y):  
        return x      # x — из окружения  
    return helper
```

- **Δ Замыкание** — функция вместе с окружением, то есть состоянием, к которому её код имеет доступ. Так прикладные языки и представляют λ-функции

```
auto screen = Screen::create(...);  
clearButton.onClick([screen]() {  
    screen.textField = "";  
});
```

$\mathbf{B} := \lambda f. \lambda g. \lambda x. f (g x)$ принимает функции:

```
def compose(f, g):  
    def helper(x):  
        return f(g(x))  
    return helper
```

- **Δ Функция высшего порядка** принимает другие функции в качестве аргументов
- ⚙ Каррировать по всем аргументам на практике не всегда удобно: `compose(f, g)` берёт сразу два

Harvard CS153, Lec 12: Functions

Законы и algebra driven design

Закодировав пару, мы получили абстракцию по данным:

- Это уже не два значения, а доменная сущность
- Реализацию можно выбрать любую, лишь бы выполнялись законы, — и дальше забыть, как она устроена
- Её можно в любой момент подменить, например на более эффективную, и пользовательский код не сломается
- Соответствие законам можно проверять автоматически

Прочитать: Sandy Maguire, *Algebra-Driven Design*; посмотреть: [John Hughes — How to specify it! \(Lambda Days 2020\)](#).

Идея кодирования по Чёрчу

- Пусть функция не возвращает результат, а принимает способ его сразу обработать: «штуку я не дам, скажи, зачем она тебе, — я всё сделаю»
- Вместо того чтобы нести объект туда, где его разберут, мы приносим разбор туда, где объект создаётся: вместо $(X) \rightarrow R$ получается $(X, (R) \rightarrow a) \rightarrow a$
- Так избавляются от аллокаций и промежуточных структур: десериализатор вместо `std::variant<Msg1, Msg2>` сразу вызывает `Impl::processMsg1(...)` или `Impl::processMsg2(...)`

Это известно как tagless final, [кодирование Чёрча — Бёма — Берардуччи](#) и CPS-преобразование; всё это ещё встретится в курсе.

Объектно-ориентированная запись пар по Чёрчу

На самом деле, приоритезировать элиминаторы - более характерно для ОО стиля.

```
interface Pair<A, B> {  
    fun <R> fold(f: (A, B) -> R): R  
}  
  
fun <A, B> pair(x: A, y: B): Pair<A, B> = object : Pair<A, B> {  
    override fun <R> fold(f: (A, B) -> R): R = f(x, y)  
}  
  
fun <A, B> fst(p: Pair<A, B>): A = p.fold { x, _ -> x }  
  
fun <A, B> snd(p: Pair<A, B>): B = p.fold { _, y -> y }
```

`object : Interface` — это конструкция для определения анонимного класса-наследника. Аналогичным образом можно сделать через именованные классы.

Объектно-ориентированная запись логических величин

```
interface Bool {  
    fun <R> myif(t: R, e: R): R  
}  
  
val tru = object : Bool {  
    override fun <R> myif(t: R, e: R): R = t  
}  
  
val fls = object : Bool {  
    override fun <R> myif(t: R, e: R): R = e  
}  
  
fun not(b: Bool): Bool = b.myif(flz, tru)
```

Объектно-ориентированная запись натуральных чисел

```
interface Nat {  
  fun <T> elim(z: T, s: (T) -> T): T  
}  
  
val zero = object : Nat {  
  override fun <T> elim(z: T, s: (T) -> T): T = z  
}  
  
fun suc(n: Nat): Nat = object : Nat {  
  override fun <T> elim(z: T, s: (T) -> T): T =  
    s(n.elim(z, s))  
}  
  
fun plus(n: Nat, m: Nat): Nat = n.elim(m, ::suc)
```

Что посмотреть в транспорте

- [Функциональное программирование как предмет и метод теоретической информатики](#)
- [Podlodka #279 — Тьюринг-полнота | Виталий Брагилевский](#)
- [Roman Elizarov: Immutability we can afford](#)
- [Памятка с полезностями от Computer Science Center](#)

Серьёзные материалы

- [Записи лекций Дениса Николаевича на МКН](#)
- Книга: Х. Барендрегт, «Ламбда-исчисление. Его синтаксис и семантика»
- [Beyond Church encoding: Böhm–Berarducci isomorphism of algebraic data types and polymorphic lambda-terms](#)